OVERVIEW

Introduction to LangChain

Building LLM-Powered Applications, Step by Step

Author: Cognizant

© 2025-2027 Cognizant | Private

CORE CONCEPTS

What is LangChain?

An open-source framework for building applications powered by LLMs

Provides standard interfaces for models, prompts, memory, tools, and retrieval

Lets you compose these pieces into "chains" instead of writing glue code from scratch

Works with multiple providers — Anthropic (Claude), OpenAI, local models via Ollama, and more

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Why Use LangChain?

Raw API calls work fine for one-off prompts, but real apps need more:

- Multi-step reasoning
- Context/memory across turns
- Tool calling and external data access
- Swappable models without rewriting app logic

LangChain standardizes these patterns so you're not reinventing them per project

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Core Building Blocks

Models

LLMs / Chat Models (e.g., Claude via langchain-anthropic)

The component that actually generates text; LangChain wraps any provider behind one consistent interface

Prompts

Reusable, parameterized prompt templates

Treat prompts like functions with inputs, not hardcoded strings, so they're easy to reuse and test

Chains (LCEL)

Pipe components together with |

Connects prompt → model → parser into a single pipeline that runs as one step

Output Parsers

Shape raw model output into structured data

Converts free-text responses into strings, JSON, or typed objects your app can use

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Core Building Blocks

Part 2 of 2

Memory

Persist conversation state across calls

Carries prior turns forward so the model has context in multi-turn conversations

Tools & Agents

Let the model call functions/APIs

Gives the model the ability to take actions, not just generate text

Retrievers

Pull in external knowledge (RAG)

Fetches relevant documents from your own data so answers aren't limited to training data

© 2025-2027 Cognizant | Private

SETUP

Setting Up Your Environment

01

Install core packages: `pip install langchain langchain-anthropic`

02

Set your API key as an environment variable

03

Recommended: isolate with `uv` or a virtual environment

04

Quick sanity check: import and instantiate a chat model

© 2025-2027 Cognizant | Private

HANDS-ON

Your First Chat Model Call

`llm = ChatAnthropic(model="claude-sonnet-4-6")` then `llm.invoke("...")`

Wraps Claude behind a consistent interface — swap providers without changing the rest of your code

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Prompt Templates

Avoid hardcoding strings — parameterize your prompts

`ChatPromptTemplate.from_template("...")` lets you inject variables safely

Encourages reuse across chains and easier testing

© 2025-2027 Cognizant | Private

CORE CONCEPTS

LangChain Expression Language (LCEL)

The `|` (pipe) operator chains components together: `chain = prompt | llm`

Each component takes input, transforms it, passes it along

Composable, readable, and supports streaming/async out of the box

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Output Parsers

Models return free text by default — parsers turn that into usable structure

Common parsers: `StrOutputParser`, `JsonOutputParser`, Pydantic-based parsers

Added at the end of a chain: `chain = prompt | llm | StrOutputParser()`

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Adding Memory

- LLM calls are stateless by default — memory reintroduces context
- Options range from simple buffer memory to summarized history
- Useful for chatbots, tutoring assistants, multi-turn workflows
- Modern approach: manage history explicitly via `RunnableWithMessageHistory`

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Tools and Agents (Preview)

Tools

Functions the model can choose to call (search, calculator, APIs)

Agents

A model + tools + a loop that decides what to do next

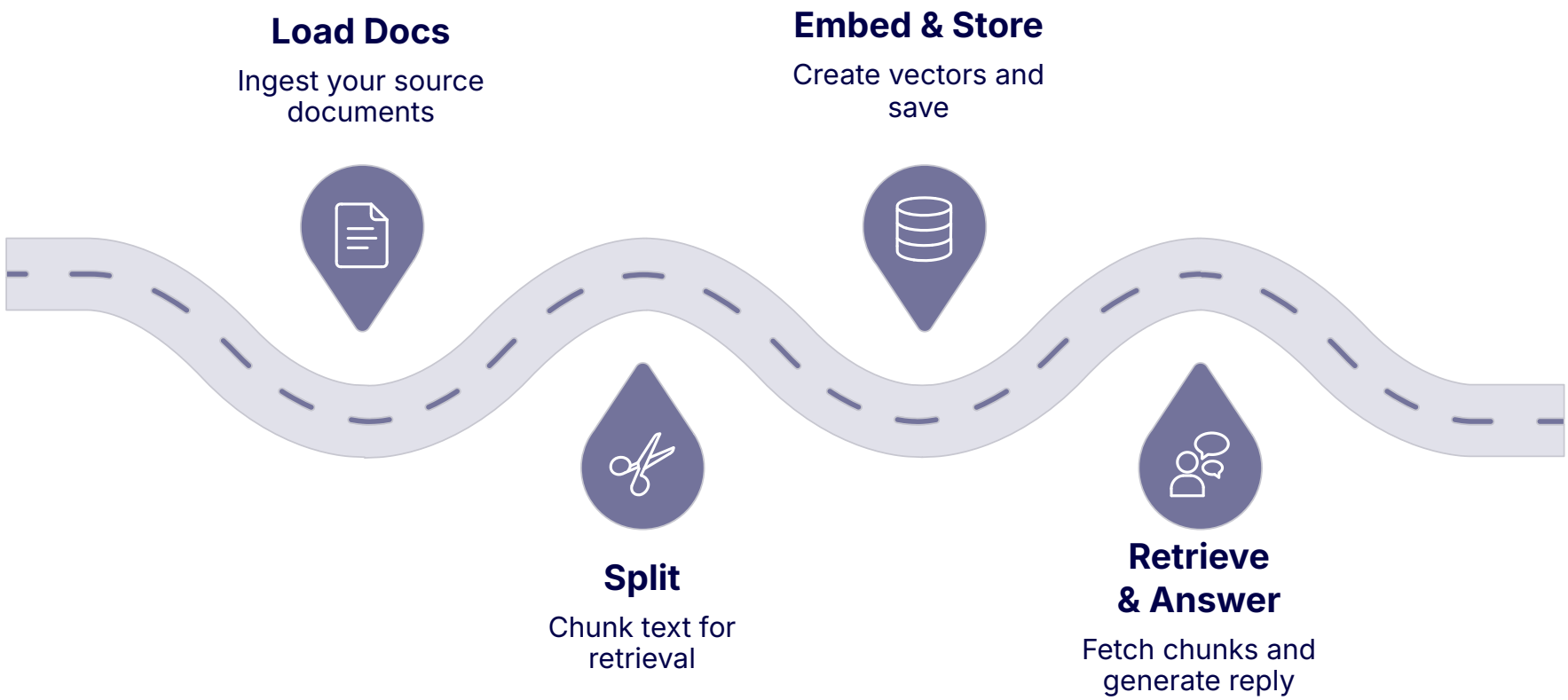
This is the foundation for autonomous, multi-step workflows

(Deeper dive in the next session)

© 2025-2027 Cognizant | Private

Retrieval-Augmented Generation (RAG) — Preview

Lets the model answer using your own documents, not just training data



LangChain provides loaders, splitters, and vector store integrations (FAISS, Pinecone, etc.)

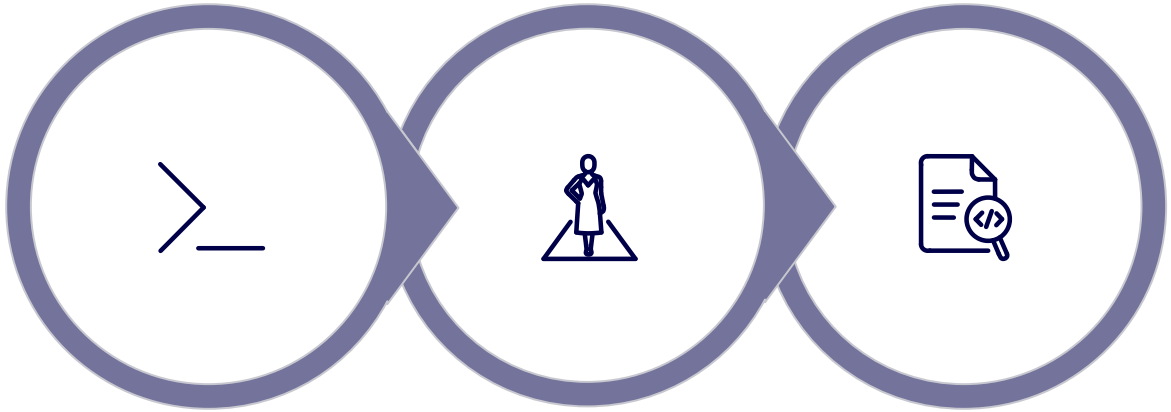
(Deeper dive in the next session)

© 2025-2027 Cognizant | Private

CORE CONCEPTS

Putting It Together: A Mini Example

One pipeline: `chain = prompt | llm | parser`



This is the full LCEL pattern: **Prompt → Model → Parser**

Nearly every chain you build will follow this shape

© 2025-2027 Cognizant | Private

BEST PRACTICES

Best Practices

1. Start simple — one prompt, one model call — before adding chains
2. Keep prompts in templates, not inline strings, for reuse and testing
3. Log intermediate outputs while debugging chains
4. Pin your LangChain version; the API evolves quickly

© 2025-2027 Cognizant | Private

HANDS-ON

What's Next

Hands-on: build your own LCEL chain with Claude

Next session: Tools, Agents, and Multi-Agent Systems

Following session: RAG pipelines with vector stores

Resources: LangChain docs, langchain-anthropic package docs

© 2025-2027 Cognizant | Private

DISCUSSION

Questions?

Let's open it up for discussion

© 2025-2027 Cognizant | Private